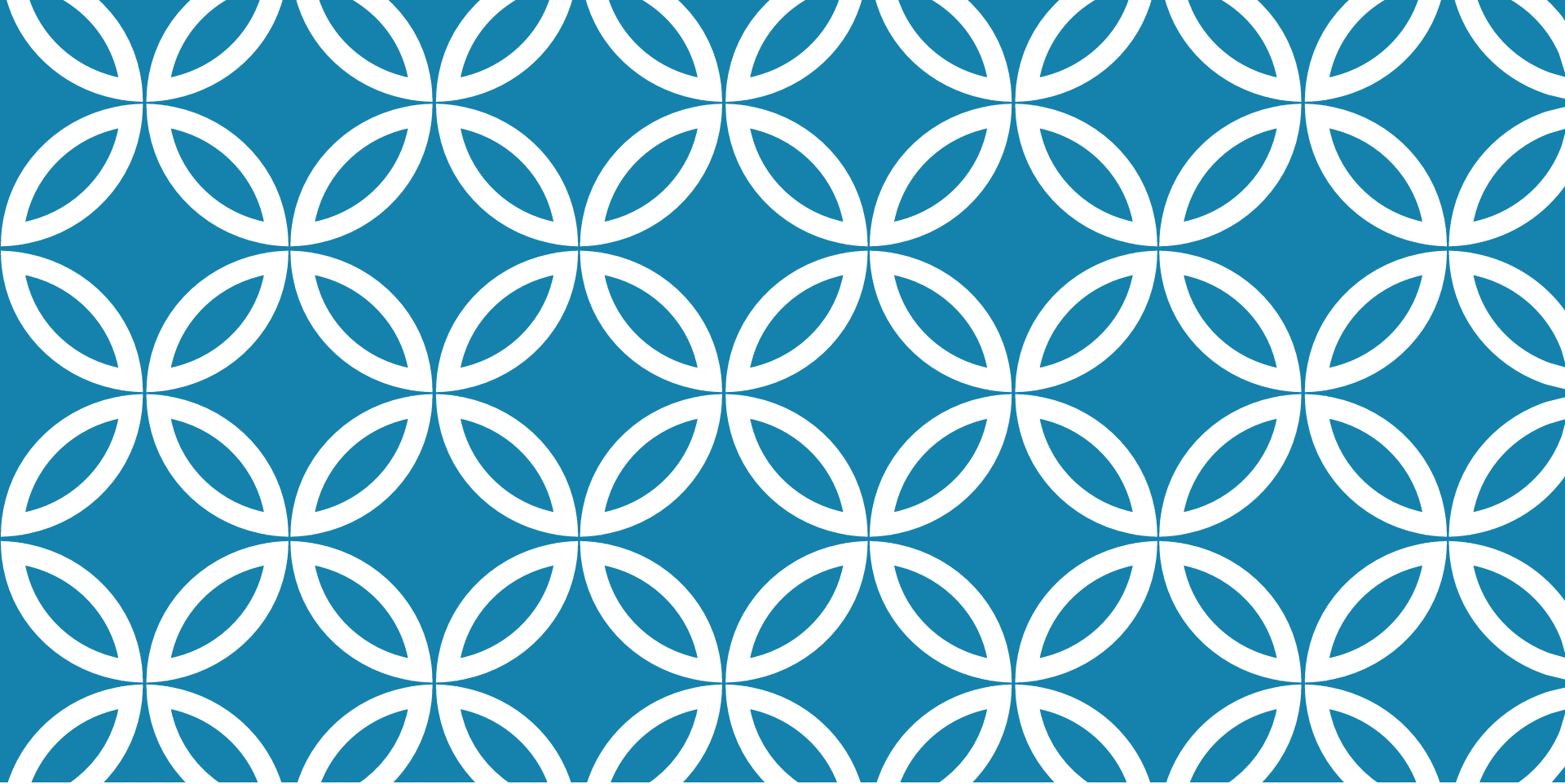




DATA ACCESS

Lecture 9

PREVIOUSLY

- Volatility driven design

CONTENT

Data Access Patterns

- Active Record
- Table Module
- DAO
- Repository
- Data Mapper

Object-Relational **Structural** Patterns

Object-Relational **Behavioral** Patterns

- Lazy Load
- Identity Map

REFERENCES

Martin Fowler et. al, Patterns of Enterprise Application Architecture, Addison Wesley, 2003 [Fowler]

Microsoft Application Architecture Guide, 2009 [MAAG]

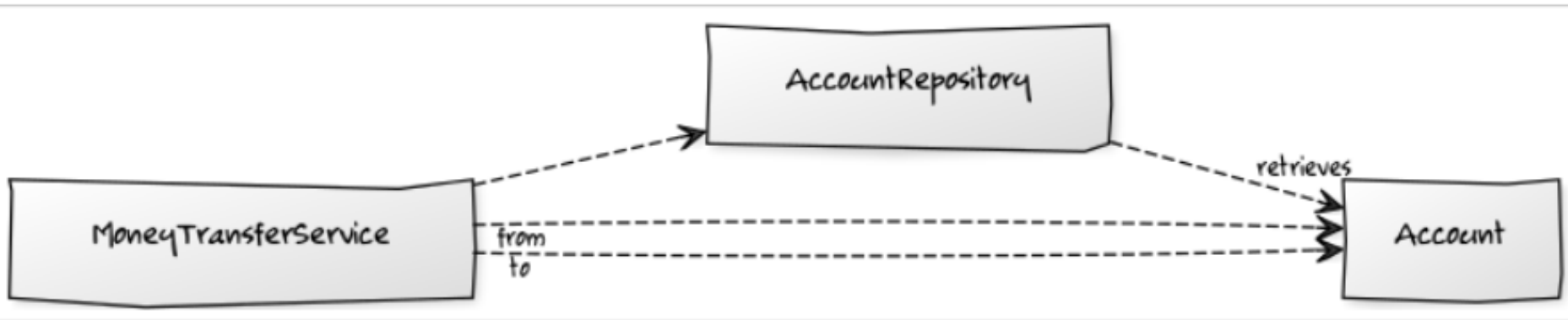
<http://jayurbain.com/msoe/se380/outline.html>

<https://medium.com/towards-polyglot-architecture/design-patterns-for-the-database-layer-7b741b126036>

FUNCTIONAL DESIGN (TRANSACTION SCRIPT)

Banking application

Money transfer functionality



```
1 public interface MoneyTransferService {
2     BankingTransaction transfer(
3         String fromAccountId, String toAccountId, double amount);
4 }
```

```

1  public class MoneyTransferServiceTransactionScriptImpl
2      implements MoneyTransferService {
3      private AccountDao accountDao;
4      private BankingTransactionRepository bankingTransactionRepository;
5      . . .
6      @Override
7      public BankingTransaction transfer(
8          String fromAccountId, String toAccountId, double amount) {
9          Account fromAccount = accountDao.findById(fromAccountId);
10         Account toAccount = accountDao.findById(toAccountId);
11         . . .
12         double newBalance = fromAccount.getBalance() - amount;
13         switch (fromAccount.getOverdraftPolicy()) {
14             case NEVER:
15                 if (newBalance < 0) {
16                     throw new DebitException("Insufficient funds");
17                 }
18                 break;
19             case ALLOWED:
20                 if (newBalance < -limit) {
21                     throw new DebitException(
22                         "Overdraft limit (of " + limit + ") exceeded: " + newBalance)
23                 }
24                 break;
25         }
26         fromAccount.setBalance(newBalance);
27         toAccount.setBalance(toAccount.getBalance() + amount);
28         BankingTransaction moneyTransferTransaction =
29             new MoneyTransferTransaction(fromAccountId, toAccountId, amount);
30         bankingTransactionRepository.addTransaction(moneyTransferTransaction)
31         return moneyTransferTransaction;
32     }
33 }

```

```
1 public enum OverdraftPolicy {
2     NEVER, ALLOWED
3 }
```

```
1 // @Entity
2 public class Account {
3     // @Id
4     private String id;
5     private double balance;
6     private OverdraftPolicy overdraftPolicy;
7     . . .
8     public String getId() { return id; }
9     public void setId(String id) { this.id = id; }
10    public double getBalance() { return balance; }
11    public void setBalance(double balance) { this.balance = balance; }
12    public OverdraftPolicy getOverdraftPolicy() { return overdraftPolicy; }
13    public void setOverdraftPolicy(OverdraftPolicy overdraftPolicy) {
14        this.overdraftPolicy = overdraftPolicy;
15    }
16 }
```

ANALYSIS

Benefits

- Simplicity

Cons

- complicated transaction logic
- duplicated logic

DOMAIN MODEL (EA PATTERN)

An object model of the domain that incorporates both behavior and data. [Fowler]

A DM creates a web of interconnected objects, where each object represents some meaningful individual, whether as large as a corporation or as small as a single line in an order form.

DM FEATURES

Business logic is organized as an OO model of the domain

- Describes both data and behavior
- Different from database model
 - Process, multi-valued attributes, inheritance, design patterns
 - Harder to map to the database

Risk of bloated domain objects

MONEY TRANSFER EXAMPLE

```
1 public class MoneyTransferServiceDomainModelImpl
2     implements MoneyTransferService {
3     private AccountRepository accountRepository;
4     private BankingTransactionRepository bankingTransactionRepository;
5     . . .
6     @Override
7     public BankingTransaction transfer(
8         String fromAccountId, String toAccountId, double amount) {
9         Account fromAccount = accountRepository.findById(fromAccountId);
10        Account toAccount = accountRepository.findById(toAccountId);
11        . . .
12        fromAccount.debit(amount);
13        toAccount.credit(amount);
14        BankingTransaction moneyTransferTransaction =
15            new MoneyTransferTransaction(fromAccountId, toAccountId, amount);
16        bankingTransactionRepository.addTransaction(moneyTransferTransaction);
17        return moneyTransferTransaction;
18    }
19 }
```

```
1 // @Entity
2 public class Account {
3     // @Id
4     private String id;
5     private double balance;
6     private OverdraftPolicy overdraftPolicy;
7     . . .
8     public double balance() { return balance; }
9     public void debit(double amount) {
10         this.overdraftPolicy.preDebit(this, amount);
11         this.balance = this.balance - amount;
12         this.overdraftPolicy.postDebit(this, amount);
13     }
14     public void credit(double amount) {
15         this.balance = this.balance + amount;
16     }
17 }
```

```
1 public interface OverdraftPolicy {
2     void preDebit(Account account, double amount);
3     void postDebit(Account account, double amount);
4 }
```

```
1 public class NoOverdraftAllowed implements OverdraftPolicy {
2     public void preDebit(Account account, double amount) {
3         double newBalance = account.balance() - amount;
4         if (newBalance < 0) {
5             throw new DebitException("Insufficient funds");
6         }
7     }
8     public void postDebit(Account account, double amount) {
9     }
10 }
```

```
1 public class LimitedOverdraft implements OverdraftPolicy {
2     private double limit;
3     . . .
4     public void preDebit(Account account, double amount) {
5         double newBalance = account.balance() - amount;
6         if (newBalance < -limit) {
7             throw new DebitException(
8                 "Overdraft limit (of " + limit + ") exceeded: " + newBalance);
9         }
10    }
11    public void postDebit(Account account, double amount) {
12    }
13 }
```

DATA ACCESS

- Communication between BLL and Data source:
 - Retrieve data from DB
 - Save data to DB
- Responsibilities:
 - Wrap the access to the DB
 - **Connect** to DB
 - **Execute** queries
 - **Map** data structure to domain model structure
 - **Store** the raw data
 - Set of records
 - Record

MONOLITHIC APPROACH

All responsibilities in one class

- Responsibilities:
 - Wrap the access to the DB
 - Connect to DB
 - Execute queries
 - + **Business Logic**
- Store the raw data
 - Set of records => **Table Module**
 - Record => **Active Record**

TABLE MODULE

Provides a single object for all the behavior on a table

Organizes domain logic with **one class per table**

Table Module has no notion of an identity for the objects that it's working with

⇒ Id references are necessary

EXAMPLE

Search countries of a specific continent

```
public class ContinentEntity {  
    private Long id;  
    private String name;  
    private String abbreviation;  
}  
  
public class CountryEntity {  
    private Long id;  
    private String name;  
    private String continentId;  
    private String abbreviation;  
}
```

EXAMPLE CONTINUED

```
public class CountryTableModule {
    private final DataSource dataSource;

    public UserTableModule(final DataSource
userDataSource) {
        this.dataSource = userDataSource;
    }

    List<CountryEntity> search(Long continentId,
String namePrefix) {
    // implementation to search in the country table based
on the search criteria
    } // ...
}
```

ACTIVE RECORD

Fowler: An object that wraps a record data structure of an external resource, such as a row in a database table, and adds some domain logic to that object.

An AR object carries both data and behavior.

Active Record is a Domain Model in which the classes match very closely the record structure of the underlying database.

CLASS OPERATIONS

- **construct** an instance of the *Active Record* from a SQL result set row
- **construct** a new instance for later insertion into the table
- static **finder** methods to wrap commonly used SQL queries and return *Active Record* objects
- methods to **update** the database and insert into the database with the data in the *Active Record*
- **getting** and **setting** methods for the fields
- methods that implement some pieces of **business logic**

EXAMPLE

```
public class Country {  
    private Long id;  
    private String name;  
    private String abbreviation;  
    // getter and setters...  
  
    public static Country fetch(Long id) {  
        // Code to interact with the repository and fetch the  
data        return country;  
    }  
    // other static methods to operate on the country record in  
the database  
}
```

INTERACT WITH THE COUNTRY OBJECT

```
Country country = new Country();  
country.setName("India");  
country.setAbbreviation("IN");  
Country.save(country);  
Country country = Country.fetch(1);
```

DISCUSSION

Key attributes

- One instance of an object represents one row of a table.
- The ORM tools commonly use it
- Encapsulates the object's data, data access logic and business logic
=> It **violates** the Single Responsibility Principle
=> It can be considered an **anti-pattern**
- It represents the table row instead of the domain object.

SEPARATING RESPONSIBILITIES

Responsibilities:

- Wrap the access to the DB
 - Connect to DB
 - Execute queries**DAO**
- Map DB structure to BL structure => **Data Mapper, Repository**
- Store the data
 - Set of records => **RecordSet**
 - Record => **DTO**

DAO PATTERN

- Intent: abstract and encapsulate all access to the data source
- Ideally: one DAO and one entity per table

Key Attributes

- It can be used for different data sources.
- It exposes one entity per table.
- It hides the database interaction complexity, i.e. via EntityManager.
- Close to the actual data store or the storage system
- Should not hold any business logic

EXAMPLE

```
public class CountryEntity {  
    private Long id;  
    private String name;  
    private String abbreviation;  
}  
  
public interface CountryDAO {  
    CountryEntity fetch(Long id);  
    Long save(CountryEntity country);  
    void update(CountryEntity country);  
    int delete(Long id);  
}
```

EXAMPLE

```
public class CountryDAOImpl implements CountryDAO {
    private final EntityManager entityManager;

    CountryEntity fetch(Long id) {
        return entityManager.find(CountryEntity.class,
id);
    }
    Long save(CountryEntity country) {
        return entityManager.persist(country);
    }
    // ...
}
```

HOW TO GET THE DAO

Strategies to get DAO's:

- Automatic DAO Code Generation Strategy
 - Metadata Mapping applied
 - JPA Hibernate
- Factory for Data Access Objects Strategy
 - Use **Abstract Factory**: When the underlying storage is subject to change from one implementation to another, this strategy may be implemented using the Abstract Factory pattern.

REPOSITORY

- Intent: bring the data from the persistent store (database) into the memory (via domain objects)
- For one aggregate class, one aggregate root should be created. Should not hold any business logic.
- **Key Attributes**
 - It encapsulates the logic or code to interact with the actual data source.
 - Might pull data from multiple data stores, aggregate them, and publish them as part of one domain object.
 - It provides a well-documented way to work with the data source.
 - It works on a domain class, i.e., Country, and not an entity.
 - It transforms data between actual data fetched from the database while publishing it via the domain object and vice versa.
 - The repositories and the aggregate root should always control the transaction boundary for the domain object.

REPOSITORY

```
public class Country {  
    private Long id;  
    private String name;  
    private String abbreviation;  
    private long population;  
}  
  
public interface CountryRepository {  
    Country fetch(Long id);  
    Long save(Country country);  
    void update(Country country);  
    int delete(Long id);  
}
```

REPOSITORY

```
public class CountryRepositoryImpl implements
CountryRepository {
    private CountryDAO countryDAO;
    private DemographDAO demographDAO;

    Country fetch(Long countryId) {
        CountryEntity countryEntity =
countryDAO.fetch(countryId);
        DemographEntity demographEntity =
demographDAO.fetch(countryId);
        // Country Mapper to copy data from the
country entity and demograph entity and prepare country
object (domain object)
    }
    Long save(Country country) {
        //prepare the entities
        return countryDAO.save(countryEntity);
    }
    // ...
}
```

DAO VS REPOSITORY

- DAO pattern should be used when interacting with one table to perform a CRUD operation
- DAO usually exposes an entity but can return a domain object for simple tables that mimic the domain object
- Repository handles a collection of domain objects
- Repository should be used where the need is to aggregate and transform the data from more than one entity and publish it as a domain object
- Repository can use DAOs but not vice versa

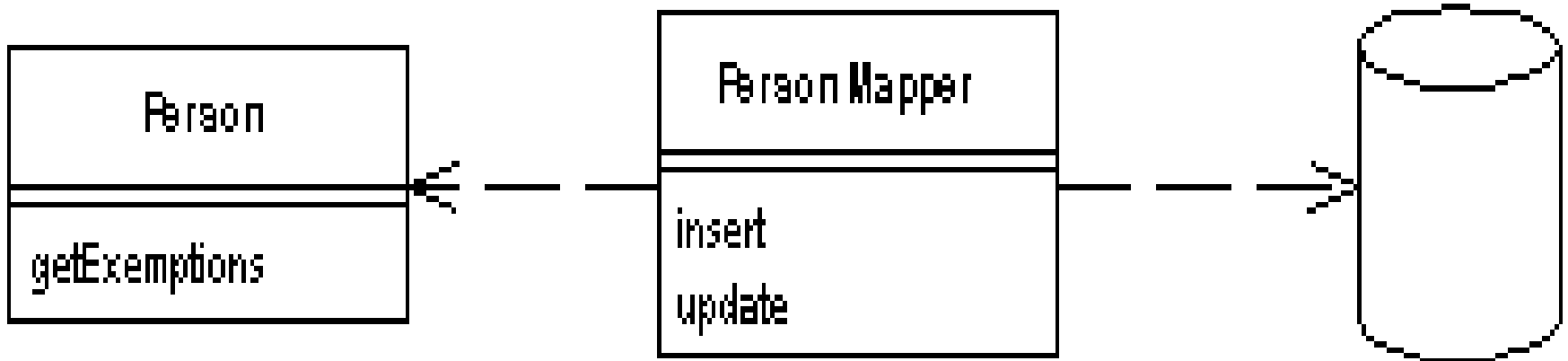
DATA TRANSFER OBJECT (DTO)

- An object that carries data between processes in order to reduce the number of method calls.
- Usually contains a set of attributes + setters and getters
- Usually aggregates data from all the server objects that the remote object is likely to want to get data from.
- DTOs are usually structured in tree structures (based on composition) containing primitive types and other DTOs
- Structured around the needs of a particular client (i.e. corresponding to web pages/GUI screens).

DATA MAPPERS

Acts as an intermediary between Domain Models and the database.

Allows Domain Models and Data Source classes to be independent of each other



DATA MAPPER LAYER ...

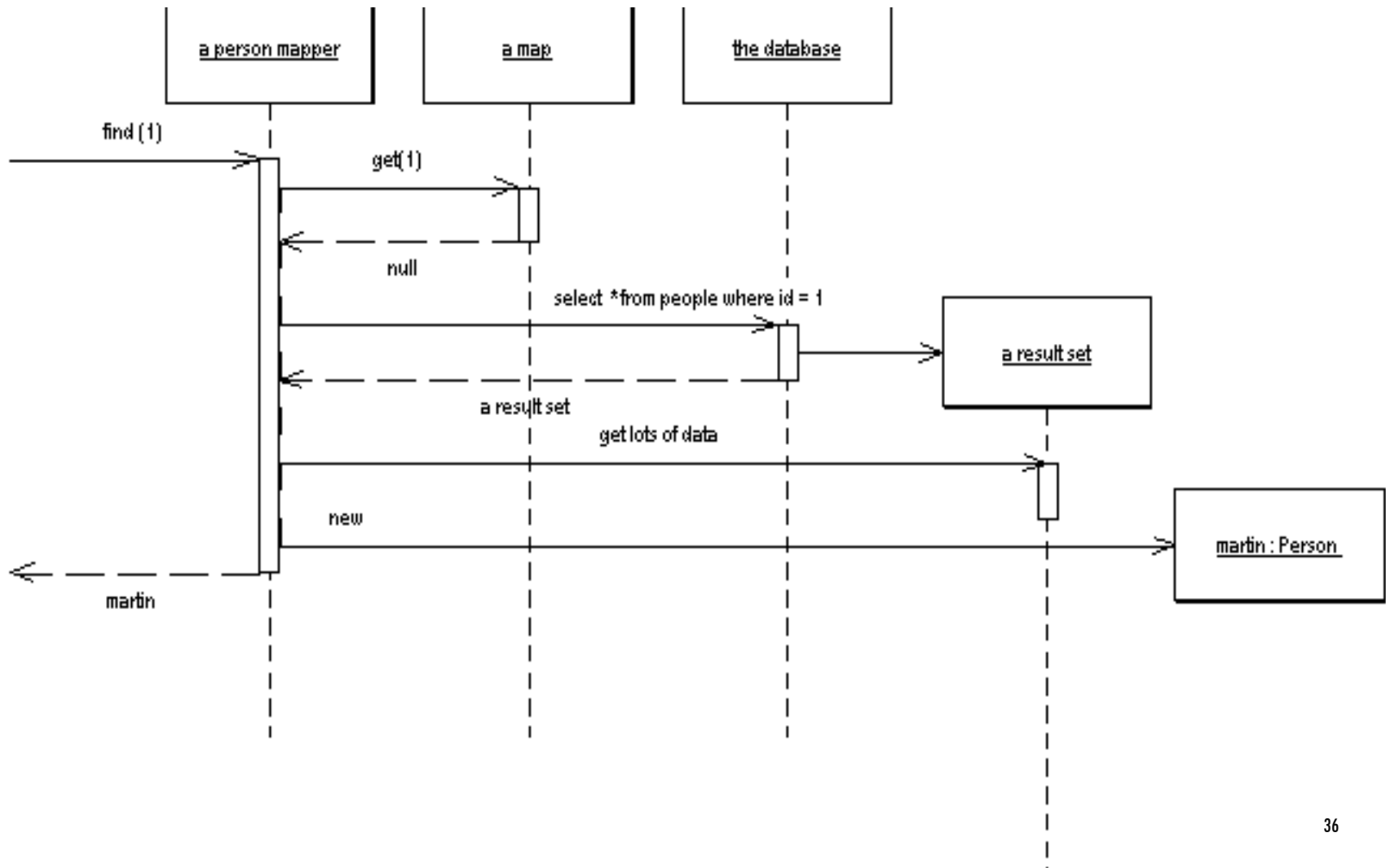
Can either

- Access the database itself, or
- Make use of a DAO.

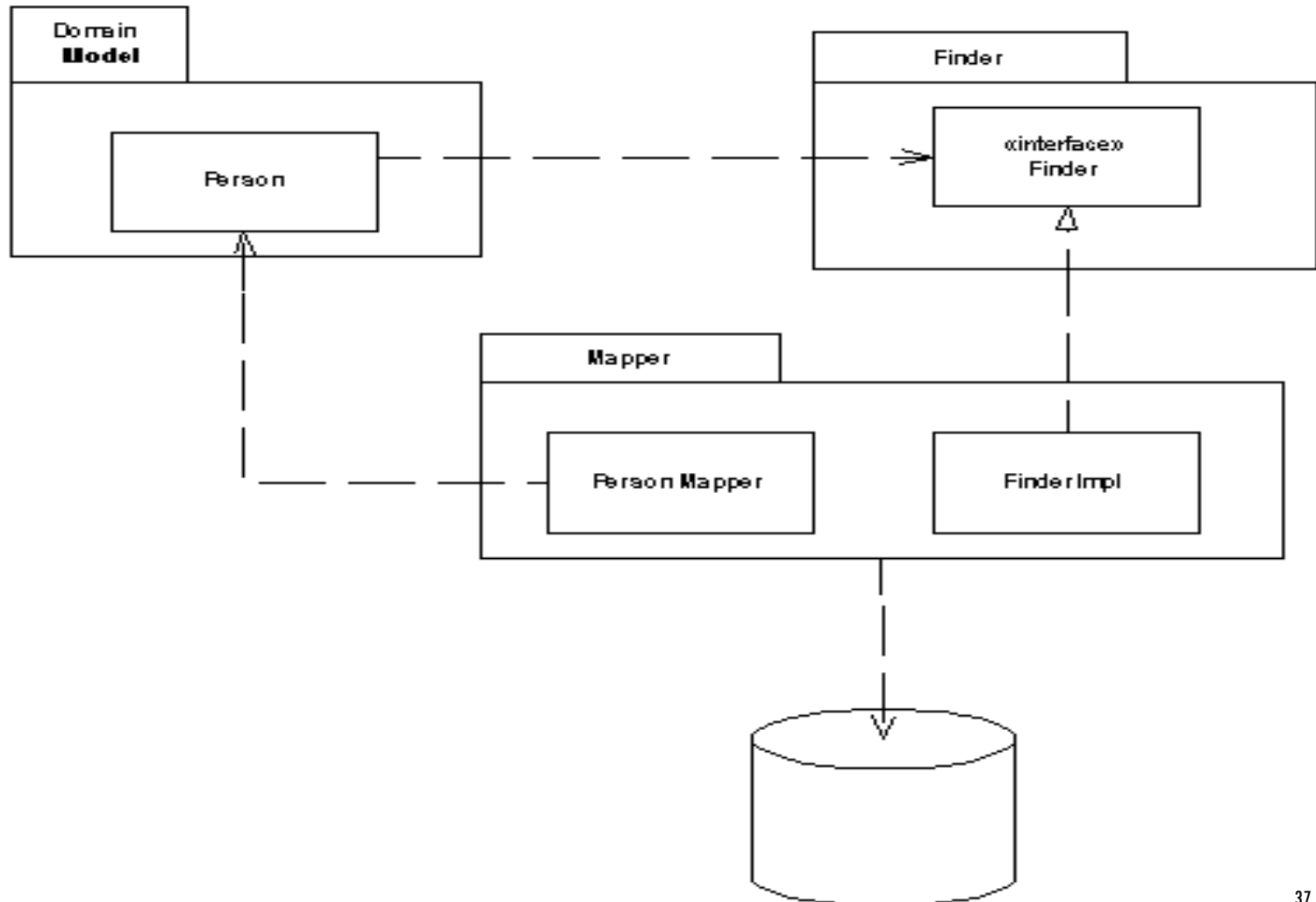
Does not contain Domain Logic.

When it uses a DAO, the Data Mapper can be placed in the (lower) BLL.

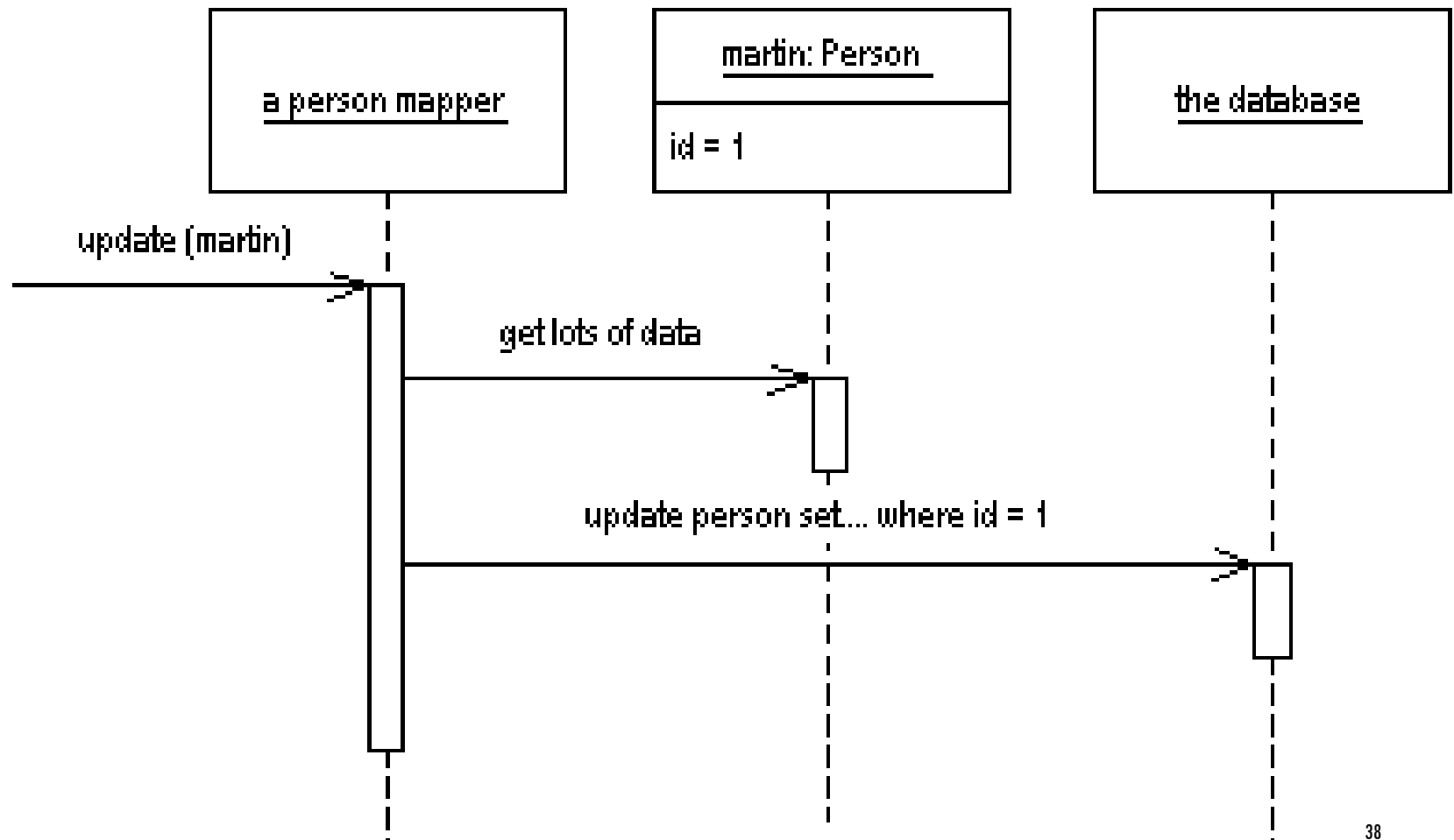
RETRIEVING DATA



FINDING OBJECTS



UPDATING DATA



FEATURES

- Independent database schema and object model
- Extra layer
- Makes sense with Domain Model

IMPLEMENTATION

```
class Person {  
    private String name;  
    private int numberOfDependents;  
    ... }
```

```
create table people (ID int primary key, lastname  
    varchar, firstname varchar, number_of_dependents int)
```


ABSTRACTMAPPER

```
class AbstractMapper...
    protected Map loadedMap = new HashMap();
    abstract protected String findStatement();
    protected DomainObject abstractFind(Long id)
    {
        DomainObject result = (DomainObject) loadedMap.get(id);
        if (result != null) return result;
        PreparedStatement findStatement = null;
        try
        {
            findStatement = DB.prepare(findStatement());
            findStatement.setLong(1, id.longValue());
            ResultSet rs = findStatement.executeQuery();
            rs.next();
            result = load(rs);
            return result;
        } catch (SQLException e)
        { throw new ApplicationException(e); }
        finally { DB.cleanUp(findStatement); }
    }
}
```

LOAD METHOD IN ABSTRACTMAPPER

```
class AbstractMapper...  
    protected DomainObject load(ResultSet rs) throws  
        SQLException  
    {  
        Long id = new Long(rs.getLong(1));  
        if (loadedMap.containsKey(id))  
            return (DomainObject) loadedMap.get(id);  
        DomainObject result = doLoad(id, rs);  
        loadedMap.put(id, result);  
        return result;  
    }  
    abstract protected DomainObject doLoad(Long id,  
        ResultSet rs) throws SQLException;
```

MAPPER CLASS IMPLEMENTS FINDER

```
class PersonMapper...
    protected String findStatement()
    {
        return "SELECT " + COLUMNS + " FROM people" + "
WHERE id = ?";
    }

    public static final String COLUMNS = " id, lastname,
firstname, number_of_dependents ";

    public Person find(Long id)
    {
        return (Person) abstractFind(id);
    }
```

DOLOAD IN PERSONMAPPER

```
class PersonMapper...  
  
protected DomainObject doLoad(Long id,  
ResultSet rs) throws SQLException  
{  
    String name = rs.getString(2)+" " +  
                  rs.getString(3);  
  
    int numDependentsArg = rs.getInt(4);  
  
    return new Person(id, name,  
numDependentsArg);  
}
```

DATA MAPPER VS REPOSITORY

Common

- both used to map data structure to domain structure

Differences

- **Level of Abstraction:** The Repository pattern abstracts the data access logic at a higher level, dealing with collections of objects, while the Data Mapper pattern abstracts the database access logic at a lower level, dealing with individual objects.
- **Responsibility:** The Repository pattern is responsible for managing collections of objects and providing a consistent API for data access, whereas the Data Mapper pattern is responsible for mapping data between the database and in-memory objects.
- **Usage:** The Repository pattern is often used in conjunction with the Data Mapper pattern. For example, a repository might use an ORM (which implements the Data Mapper pattern) to interact with the database.

STRUCTURAL PROBLEMS

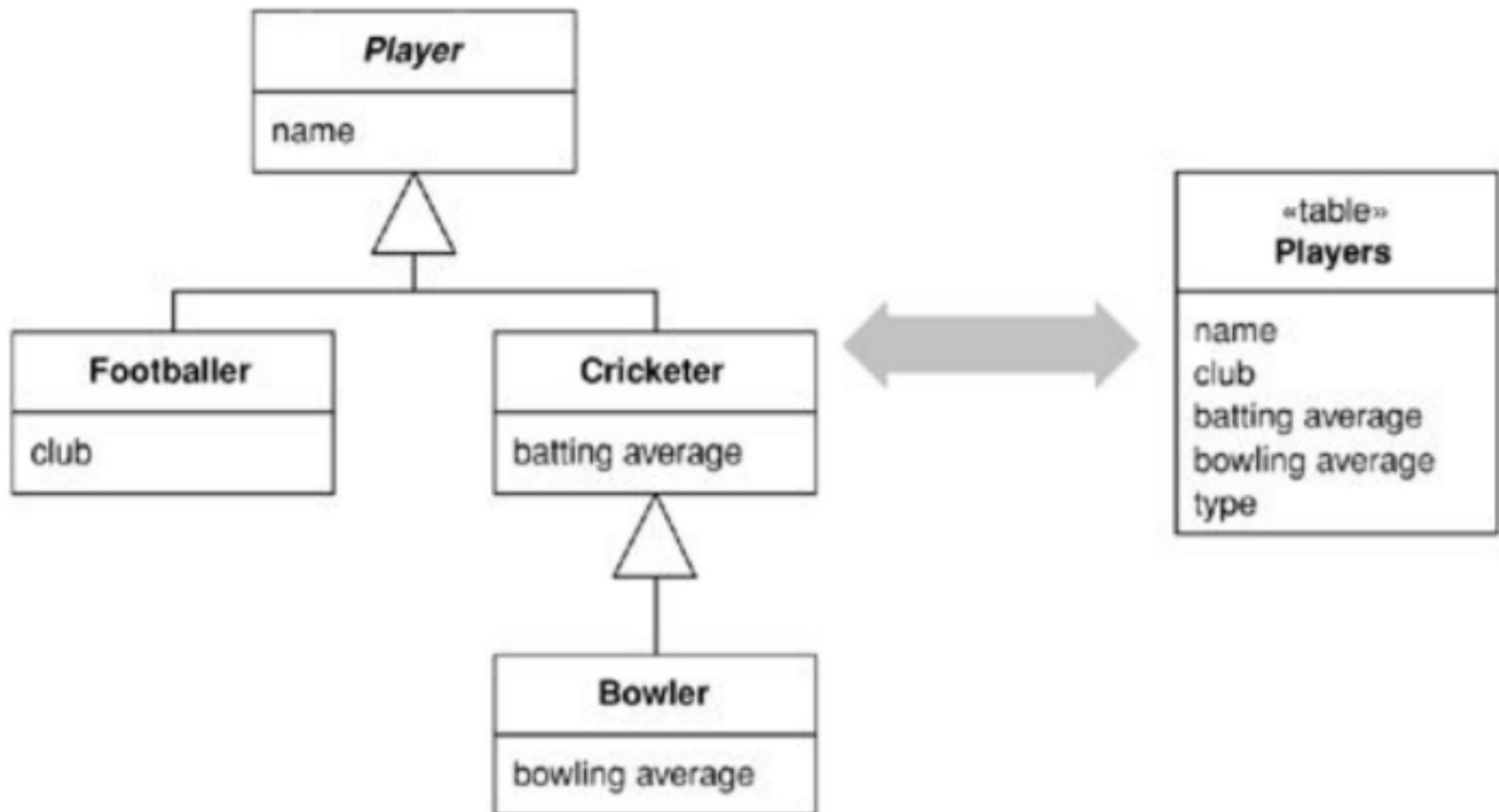
Object Relationships structural mapping problems

- Association/Composition
- **Inheritance**

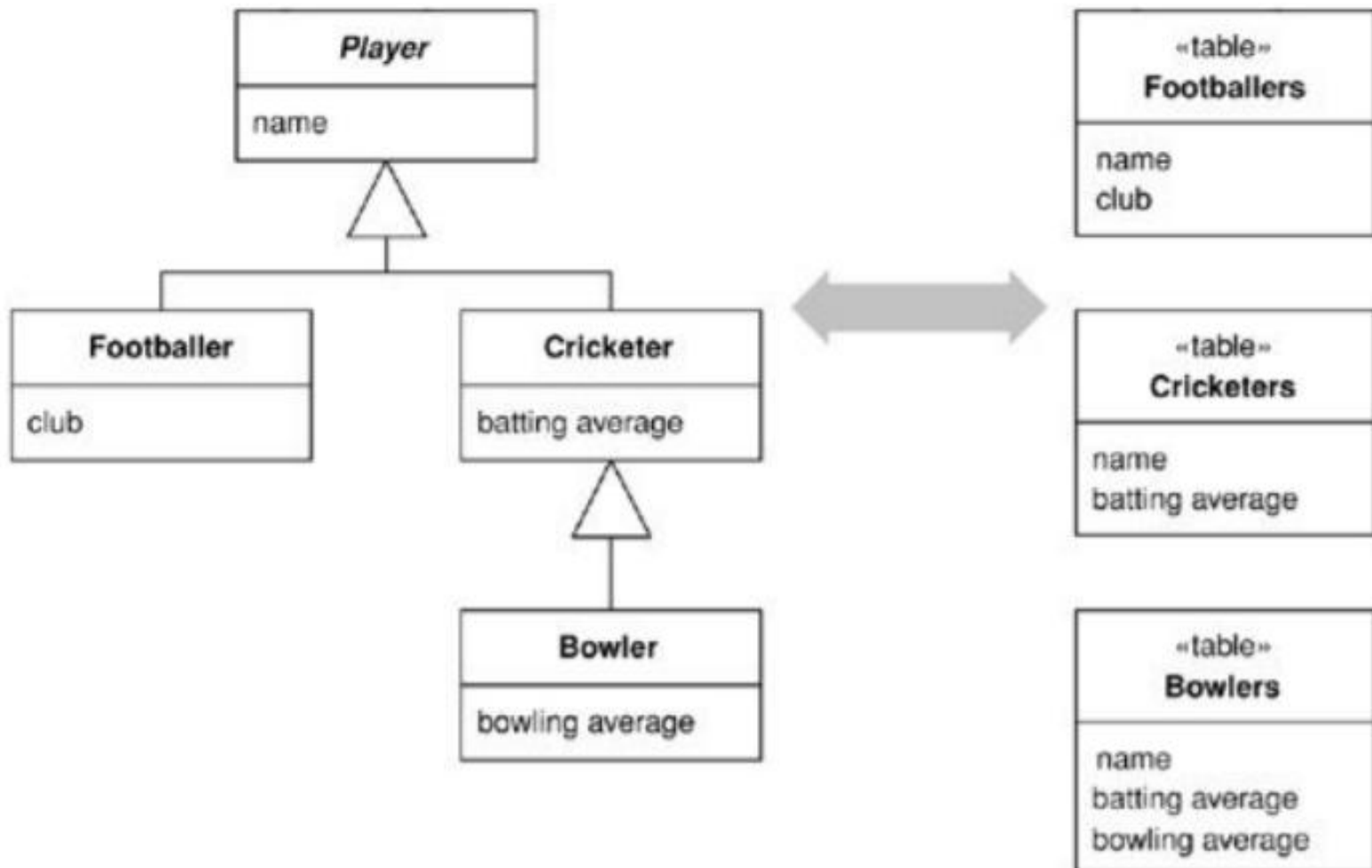
Object-Relational Structural Patterns

- **Single Table Inheritance**
- **Class Table Inheritance**
- **Concrete Table Inheritance**

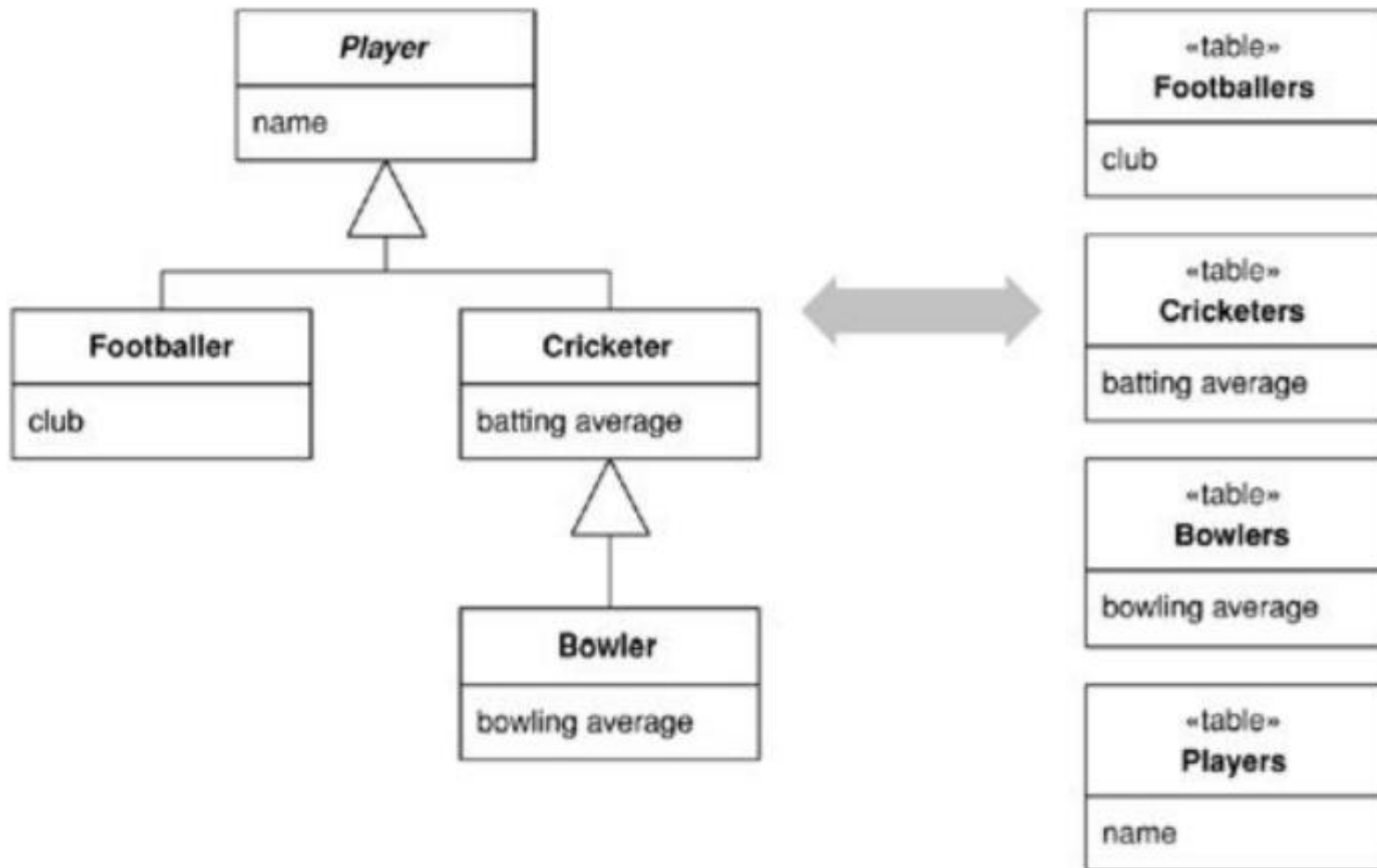
SINGLE TABLE INHERITANCE



CONCRETE TABLE INHERITANCE



CLASS TABLE INHERITANCE



BEHAVIORAL PROBLEMS

Networks of objects

- E.g. Invoice heading relates to invoice details
- Invoice details refers to Products
- Products refers to Suppliers
- ...

What to do?

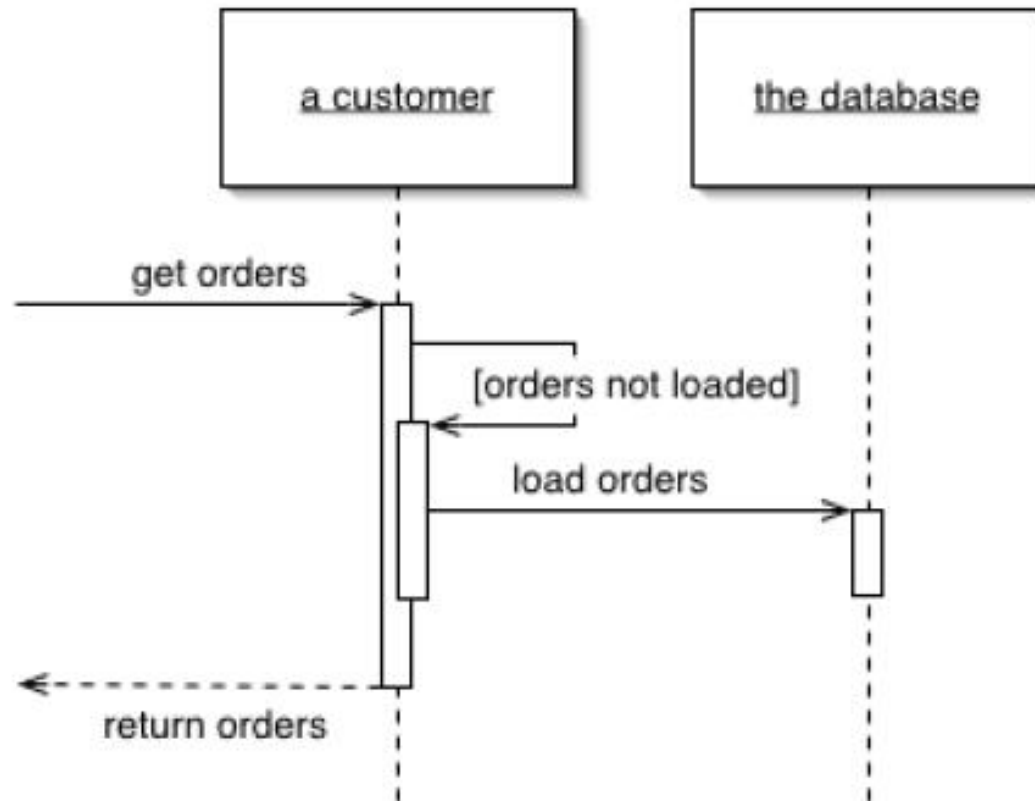
- Load them all into memory?
- How to disallow multiple in-memory copies

Object-Relational Behavioral Patterns

- **Lazy Load**
- **Identity Map**

LAZY LOAD

An object that doesn't contain all of the data you need but knows how to get it.



IMPLEMENTATION OPTIONS

Lazy initialization: every access to the field checks first to see if it's null. If so, it calculates the value of the field before returning the field.

```
class Supplier...  
  
    public List getProducts() {  
        if (products == null) products = Product.findForSupplier(getID());  
        return products;  
    }
```

Virtual proxy: looks like the object but doesn't contain anything. Only when one of its methods is called does it load the object from the database.

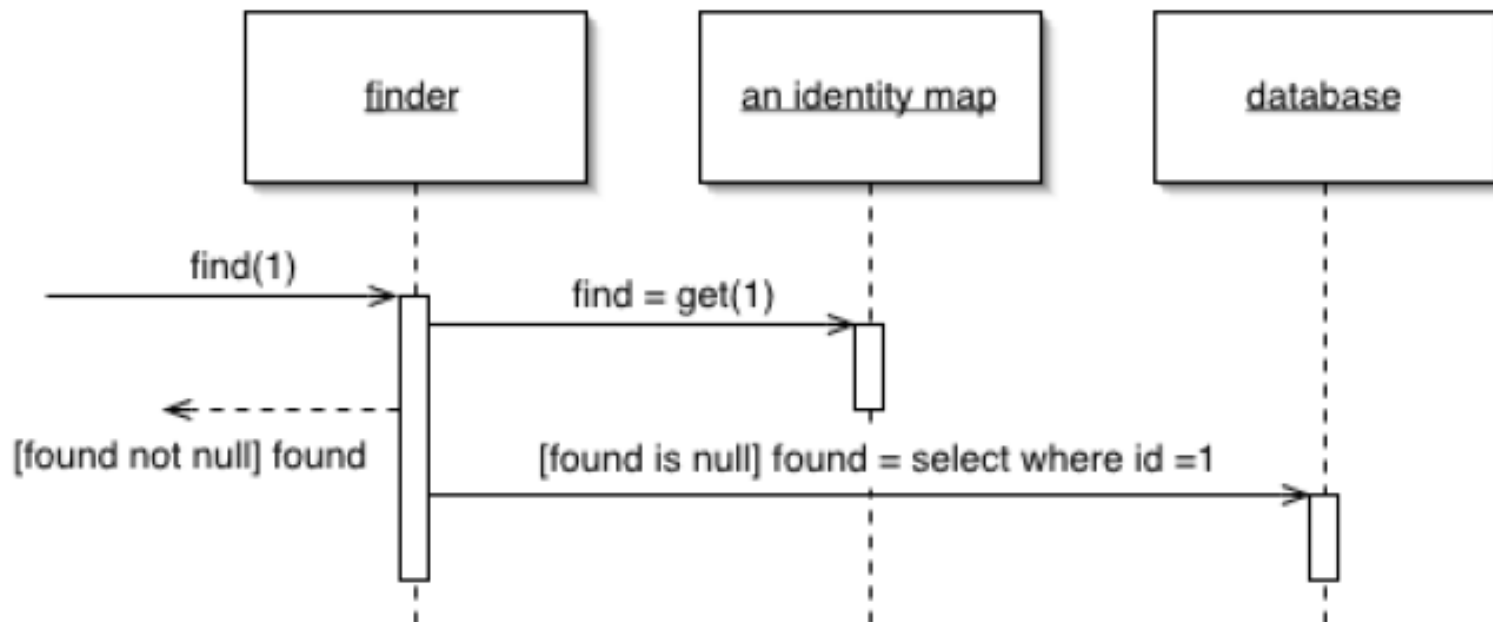
Ghost: the real object in a partial state. Ex. when you load the object from the database it contains just its ID. Whenever you try to access a field it loads its full state.

DISCUSSION

- Inheritance might be a problem with Lazy Load
- Can easily cause more database accesses than you need (ex. fill a collection with Lazy Loads)
- Deciding when to use Lazy Load is all about deciding how much you want to pull back from the database as you load an object, and how many database calls that will require.

IDENTITY MAP

Ensures that each object gets loaded only once by keeping every loaded object in a map. Looks up objects using the map when referring to them



HOW IT WORKS

Map key?

- Primary key in the table (if it is a single column and immutable)

Explicit vs. generic

- `findPerson(1)`
- `find ("Person", 1)`

How many?

- One map/session (if you have database-unique keys)
- One map/table
- One map/class
- One map/inheritance tree

WHEN TO USE IT

- Use an Identity Map to manage any Entity object brought from a database and modified.
- Acts as a cache for database reads
- May not need an Identity Map for immutable objects
- Helps avoid update conflicts within a single session, but it doesn't do anything to handle conflicts that cross sessions

WRAP-UP

- The data access layer involves several responsibilities
- Different patterns to distribute these responsibilities across components
- Domain model vs. (relational) data model
 - Structure can be different
 - Relationship mapping
 - Association
 - Inheritance